MlasQuantizeLinear 多平台向量实现对比与RVV扩展指令建议

概述

分析目标: `MlasQuantizeLinear` — 线性量化（float32→int8/uint8/int16/uint16） **算法**: `Output = Saturate(RoundToEven(Input / Scale) + ZeroPoint)` **基准实现**: RVV VLEN=512, e32m2配置 (VL=32 float32) **分析平台**: x86 AVX/SSE2, ARM NEON64, LoongArch LSX, Power VSX, S390X **BBV数据**: 基于QEMU-BBV profiling on 独立测试可执行文件 (output/bbv_rvv512/quantize-linear/)

指令方案汇总

优先级	扩展指令	来源平台	BB内收益	实现难度	RVV现状
P0	vfcvt_rne_x_f.v (饱和窄化融合)	ARM NEON vcvtnq + vqmovn	量化BB内减少约25% （int32→int8窄化）	中	RVV需3步窄化: vncvt i32→i16 + vncvt i16→i8
P1	vqshrn.v (饱和右移窄化)	ARM NEON vqmovun_s16	uint8输出BB内减少约20%	高	RVV无饱和窄

优先级	扩展指令	来源平台	BB内收益	实现难度	RVV现状
					化，需手动clamp + vncvt

> **注**: 原*P2*近似倒数(*vfrecip.v*)已移除。*RVV base V extension*已提供 `vfrec7.v` (*7-bit*近似倒数) , *intrinsic:* `vfloat32m1_t __riscv_vfrec7(vfloat32m1_t vs2, size_t vl)`，功能等价于*x86* `rcpps` 和*ARM* `vrecpeq_f32`。

收益计算方式（基于BBV profiling数据）： - BB内收益 = (原BB指令数 - 扩展后BB指令数) / 原BB指令数 × 100% - 整体收益结合BBV profiling热点占比计算，数据来自 `output/bbv_rvv512/quantize-linear/`

基准RVV实现分析

RVV实现结构 (VLEN=512, e32m2, VL=32)

```
// rvv_quantize_linear.inl U8核心循环
while (avl > 0) {
    size_t vl = __riscv_vsetvl_e32m2(avl);           // 1 setvl

    vfloat32m2_t v_input = __riscv_vle32_v_f32m2(Input, vl);     // 1 加载
    vfloat32m2_t v_scaled = __riscv_vfdiv_vf_f32m2(v_input, Scale, vl); // 1 除法
    vfloat32m2_t v_clamped = __riscv_vfmax_vf_f32m2(v_scaled, MinVal, vl); // 1 clamp m
```

```
    v_clamped = __riscv_vfmin_vf_f32m2(v_clamped, MaxVal, vl);             // 1 clamp m

    vint32m2_t v_rounded = __riscv_vfcvt_x_f_v_i32m2(v_clamped, vl); // 1 舍入转换
    vint32m2_t v_zp = __riscv_vmv_v_x_i32m2((int32_t)ZeroPoint, vl); // 1 零点广播
    vint32m2_t v_int = __riscv_vadd_vv_i32m2(v_rounded, v_zp, vl);   // 1 加零点

   // 窄化链: int32m2 → uint16m1 → uint8mf2
    vuint32m2_t v_u32 = __riscv_vreinterpret_v_i32m2_u32m2(v_int);     // 0 重解释
    vuint16m1_t v_u16 = __riscv_vncvt_x_x_w_u16m1(v_u32, vl);        // 1 窄化32→16
    vuint8mf2_t v_u8 = __riscv_vncvt_x_x_w_u8mf2(v_u16, vl);         // 1 窄化16→8
    __riscv_vse8_v_u8mf2(Output, v_u8, vl);                            // 1 存储
}
// 主循环: 10条指令/32元素 (U8)
```

RVV指令计数分析

操作阶段	指令数	说明
数据加载	1	vle32 (e32m2)
量化计算	5	vfdiv + vfmax + vfmin + vfcvt + vadd
窄化链 (→8bit)	2	vncvt 32→16 + vncvt 16→8
窄化链 (→16bit)	1	vncvt 32→16
数据存储	1	vse8/vse16
总计 (U8)	**10**	
总计 (U16/S16)	**9**	

关键观察： - U8输出需要**两步窄化**（int32→int16→int8），这是RVV的必要LMUL规则 - `vfcvt_x_f` 执行round-to-nearest-even，与ONNX规范一致 - 除法 `vfdiv` 是单条指令，但在硬件上可能需要多周期 - 当前实现依赖clamping确保值在范围内，无需显式饱和

多平台实现对比

ARM NEON64 实现

```
// NEON MlasQuantizeLinearVector
FloatVector = MlasDivideFloat32x4(FloatVector, ScaleVector);  // 1 div
FloatVector = vmaxnmq_f32(FloatVector, MinimumValueVector);   // 1 clamp min
FloatVector = vminnmq_f32(FloatVector, MaximumValueVector);   // 1 clamp max
auto IntegerVector = vcvtnq_s32_f32(FloatVector);             // 1 round+convert
IntegerVector = vaddq_s32(IntegerVector, ZeroPointVector);    // 1 add zp

// PackBytes<uint8_t>: 2步窄化
uint16x8_t WordVector = vreinterpretq_u16_s32(IntegerVector);
WordVector = vuzp1q_u16(WordVector, WordVector);              // 1 shuffle
uint8x16_t ByteVector = vreinterpretq_u8_u16(WordVector);
ByteVector = vuzp1q_u8(ByteVector, ByteVector);               // 1 shuffle
// 存储通过vst1q_lane_s32
```

NEON优势: - `vcvtnq` 原生round-to-nearest-even - `vuzp1q` 做高效narrowing pack（本质是shuffle + 截断） - `vqmovn_s32` 支持饱和窄化（int32→int16 with saturation）

x86 AVX/SSE2 实现

```
// SSE2 MlasQuantizeLinearVector
FloatVector = _mm_div_ps(FloatVector, ScaleVector);    // 1 div
FloatVector = _mm_max_ps(FloatVector, MinimumValueVector); // 1 clamp
FloatVector = _mm_min_ps(FloatVector, MaximumValueVector); // 1 clamp
auto IntegerVector = _mm_cvtps_epi32(FloatVector);    // 1 round+convert
IntegerVector = _mm_add_epi32(IntegerVector, ZeroPointVector); // 1 add

// PackBytes<uint8_t>
IntegerVector = _mm_packus_epi16(IntegerVector, IntegerVector); // 1 饱和pack
IntegerVector = _mm_packus_epi16(IntegerVector, IntegerVector); // 1 饱和pack
```

SSE2优势: - `_mm_packus_epi16` 原生饱和窄化（int16→uint8 with saturation） - `_mm_packs_epi32` 有符号饱和窄化（int32→int16 with saturation） - 两步pack即可完成 int32→uint8

LoongArch LSX 实现

```
// LSX MlasQuantizeLinearVector
FloatVector = __lsx_vfdiv_s(FloatVector, ScaleVector);
FloatVector = __lsx_vfmax_s(FloatVector, MinimumValueVector);
FloatVector = __lsx_vfmin_s(FloatVector, MaximumValueVector);
auto IntegerVector = __lsx_vftint_w_s(FloatVector);          // round-to-nearest
IntegerVector = __lsx_vadd_w(IntegerVector, ZeroPointVector);

// PackBytes<uint8_t>: 需要手动饱和 + 窄化
__m128i zero = __lsx_vldi(0);
tmp = __lsx_vmax_h(integervector, zero);     // clamping
tmp2 = __lsx_vsat_hu(tmp, 7);               // 无符号饱和到8bit
integervector = __lsx_vpickev_b(tmp2, tmp2); // 窄化pack
// 需要两组同样的操作
```

LSX特点: - `__lsx_vsat_hu` 提供无符号饱和到指定bit宽度 - `__lsx_vpickev_b` 做窄化pack - 比RVV多需要显式clamping步骤

Power VSX (POWER10) 实现

```
// Power VSX: 使用vec_round + vec_signed + vec_pack序列
FloatVector = vec_round(FloatVector);               // round
auto IntegerOutVector = vec_signed(FloatVector);    // float→int
IntegerOutVector = vec_add(IntegerOutVector, ZeroPointVector);
auto ShortVector = vec_pack(IntegerOutVector0, IntegerOutVector1); // 饱和pack
auto CharVector = vec_pack(ShortVector0, ShortVector1);
```

VSX特点: - `vec_pack` 提供饱和窄化 - `vec_round` 独立round指令

指令方案详细分析

P0: 饱和窄化融合指令 (vfcvt_rne_x_f + vnarrow_sat)

来源: ARM NEON `vcvtnq` + `vqmovn` / `vqmovun`, x86 SSE2 `_mm_packus_epi16`

当前RVV瓶颈： - U8输出：需要 `vncvt i32→i16` + `vncvt i16→i8` = 2条窄化指令 - 无饱和窄化：需预先通过vfmax/vfmin clamp到目标范围 - 实际上clamping已经在float域完成（在vfcvt之前），所以窄化阶段的值已在范围内

建议融合方案:

```
// 当前: vfcvt_x_f + vadd + vreinterpret + vncvt + vncvt + vse  (6条)
// 融合: vfcvt_rne_sat_x_f.v  (round+convert+clamp, 1条替代vfcvt+clamping)
//       vnarrow_sat_i32_i8   (int32→int8 饱和窄化, 1条替代2步vncvt)
// 总计: 4条 (减少33%)
```

BB内收益估算: 约25%（减少3条指令/10条→7条）

实现难度: 中。需要新指令定义，但语义清晰（融合已有操作）。

P1: 向量饱和窄化指令 (vqshrn/vnarrow_sat)

来源: ARM NEON `vqmovn_s32` / `vqmovun_s16`, x86 `_mm_packs_epi32` / `_mm_packus_epi16`

当前RVV瓶颈: - `vncvt_x_x_w` 是截断窄化（直接取低半部分），不保证饱和 - 当值超出目标范围时需要预先clamp - U8场景：需要int32→[0,255] clamp → uint16 → uint8 两步

建议指令:

```
vnarrow_sat_i32_i8  — int32→int8 饱和窄化 (1条替代2步vncvt)
vnarrow_sat_u32_u8  — uint32→uint8 饱和窄化
```

```
vnarrow_sat_i32_i16 — int32→int16 饱和窄化 (1条替代1步vncvt + clamp)
vnarrow_sat_u32_u16 — uint32→uint16 饱和窄化
```

BB内收益估算: U8场景约20%（减少2条指令/10条→8条），实际上因为clamping已在float域完成，主要减少的是窄化步数

实现难度: 高。需要定义新语义，涉及多宽度跨LMUL操作。

与其他量化实现对比总结

特性	RVV	NEON	SSE2	LSX
Round-to-nearest-even	vfcvt_x_f	vcvtnq	_mm_cvtps_epi32	vftint_w
饱和窄化 int32→int16	❌ 需 clamp+vncvt	✅ vqmovn_s32	✅ _mm_packs_epi32	✅ vsat_w+
饱和窄化 int16→uint8	❌ 需 clamp+vncvt	✅ vqmovun_s16	✅ _mm_packus_epi16	✅ vsat_hu+vp
窄化步数 (→uint8)	2步	2步(shuffle)	2步(pack)	2步(sat+pa
转换+窄化融合	❌	❌	❌	❌

关键结论： - RVV的窄化步数与其他平台相同（2步→uint8），但缺少饱和窄化 - 由于clamping已在float域完成（vfcvt_x_f之前），饱和窄化在QuantizeLinear场景下收益有限 - **真正的瓶颈**是vfdiv的延迟（如果编译器未能优化为乘法）和vfcvt_x_f的延迟 - 相比其他平台，RVV使用e32m2配置（VL=32），处理吞吐量更高

建议优先级

优先级	建议	预期收益	难度
P0	编译器优化：常量Scale → 乘法 (vfdiv→vfmul)	高（消除除法延迟）	低（编译器已有）
P1	饱和窄化指令 (vnarrow_sat)	中（减少窄化步数，提高鲁棒性）	高

实际收益评估：QuantizeLinear仅占2.99%执行时间，即使全部优化也仅减少整体约0.5-1%的执行时间。优化优先级低于QGEMM (73.97%)、Logistic (10.01%)、QuickGelu (9.63%) 和 ReduceMinMax (4.68%)。

审查日志

版本	日期	修改内容
R1	2026-04-26	初始报告生成
R2	2026-04-26	基于BBV profiling数据更新热点占比和收益估算
R3	2026-04-26	**移除P2 vfrecip.v**：RVV base V extension已提供 `vfrec7.v`（7-bit近似倒数），原报告错误声称RVV无近似倒数指令。更新BBV数据来源描述，修正收益计算说明。